

南京大学



软件架构设计研究

2024 春-软件系统设计-架构部分-作业 2

组长	李薛成	211850016
组员	张铭铭	211250234
	陈皓鑫	211250236
	张哲恺	211250245

June 2, 2024

Task 1: ADD 方法应用报告

本报告研究的系统为银行对账单系统。该系统的核心流程是读取原对账单系统的数据，经过验证后生成待外部供应商提供数字签名的新对账单，并将其分发给对应的供应商，并在收到完成签名的对账单后将其存储到数据库中。

1 迭代一

1.1 步骤 1: 确认有足够的需求信息

根据描述，该银行对账单系统的约束如下：

- 每一个用户的账户和权限需要由现有的、银行各种业务都在使用的用户目录服务器处理。
- 与原数据源的通信需要通过 JDBC 实现。
- 与数字签名系统的通信需要通过 Web 服务完成，通信过程中发送和返回的信息需要使用符合政府制定的规范的 XML 数据格式。
- 该系统需要能且仅能被在银行内网中的用户通过浏览器访问。

系统主要关注以下几个质量属性：可靠性、性能、可用性、安全性。

- **可靠性**：在正常运行条件下，系统批处理流程需要以 100% 的完成度完整执行。
- **性能**：在正常运行条件下批处理流程启动后，读取、处理并向签名提供商发送 200 万份银行对账单需要的时间不超过一个小时。
- **可用性**：在正常运行时，系统也可能会在诸如读取数据源、向数据签名系统发送信息等过程中出现故障。此时系统需要向管理员发送通知，由管理员手动重启流程，在流程重启后，系统需要仅处理尚未完成的对账单信息。
- **安全性**：系统需要将系统在任何时刻执行的任何操作，全部记录在操作日志中。

1.1.1 可靠性

Element	Statement
Source	定时系统、系统管理员
Stimulus	定期自动触发执行流程、管理员手动重启流程
Artifact	银行对账单系统，主要是读取、验证信息并生成对账单的子系统
Environment	正常运行环境、运行故障环境
Response	系统执行对账单处理流程、系统尝试重新执行对账单处理流程
Response Measure	1. 在正常运行条件下，系统需要以 100% 的完成度执行处理流程 2. 发生故障后，管理员手动重启并完成所有任务的次数需要不超过 2 次

1.1.2 性能

Element	Statement
Source	定时系统、数据源、存储系统
Stimulus	定期自动触发执行流程、子系统通信故障
Artifact	银行对账单系统
Environment	正常运行环境、运行故障环境
Response	系统执行对账单处理流程、系统分析故障原因并尽可能快速自主恢复
Response Measure	1. 在正常运行条件下，系统生成发送 200 万份对账单需要的时间不超过 1 小时 2. 发生故障后，如果系统分析后判断该问题不需要人工介入，通过简单重试就可以解决，则不断重试的间隔需要不大于 1s

1.1.3 可用性

Element	Statement
Source	系统管理员
Stimulus	读取源数据或发送对账单时出现故障
Artifact	银行对账单系统
Environment	运行故障环境
Response	向管理员发送通知，提醒管理员手动重启流程
Response Measure	1. 每次系统故障时都需要保证将通知发送给系统管理员 2. 每次系统故障重启流程后都需要保证所有数据都被处理完毕 3. 每次系统故障重启时都需要保证任务队列中只包含故障时未完成处理的任务和未开始处理的任务

1.1.4 安全性

Element	Statement
Source	用户、定时系统、系统管理员
Stimulus	上传数字签名、定期自动触发执行流程、手动重启流程等
Artifact	银行对账单系统整体系统
Environment	正常运行环境、运行故障环境等
Response	系统将所有执行的操作和状态记录在系统日志中
Response Measure	1. 系统需要将任何操作和状态都记录到系统日志中

1.2 步骤 2：选择要分解的系统元素

在迭代一，我们选择**银行对账单系统**作为要分解的系统元素。

1.3 步骤 3：识别候选的架构驱动要素

#	架构驱动要素	重要性	难度
1	限制一：使用现有用户服务器	高	低
2	限制二：使用 JDBC 连接数据库	高	低
3	限制三：使用 WEB 服务与数字签名提供系统通讯	高	中
4	限制四：系统提供 WEB 服务供内网通过浏览器进行访问	高	中
5	质量属性一：正常情况下完整完成批处理任务	高	中
6	质量属性二：至多在一小时内生成并发送 200 万对账单	高	高
7	质量属性三：故障时记录状态并能在重启后恢复执行	高	高
8	质量属性四：将所有操作存入日志中	高	中
9	用例一：用户可以按条件筛选、指定重新处理对账单	中	中
10	用例二：用户可以登入系统	高	低
11	用例三：用户可以指定系统根据处理过程生成报告	低	低
12	用例四：管理员可以查询、按条件筛选用户日志	高	中

1.4 步骤 4：选择符合架构驱动的设计概念

1.4.1 步骤 4.1 识别设计关注点

设计关注点	子关注点
提升系统处理效率	提升系统资源利用率
	提升数据库读写效率
保证正常情况下的任务完成率	引入任务调度与监控机制
具备应对故障的能力	引入故障预防机制
	引入故障检测机制
	引入故障恢复机制
保证系统的安全性	保证日志对操作的覆盖率

1.4.2 步骤 4.2 & 4.3 列出并选择策略

1.4.2.1 提升系统资源利用率的备选策略

#	策略	资源使用	性能提升
1	采用多线程编码软件系统	单个服务器节点的单个进程	小
2	采用 Map Reduce 框架编码软件系统	一个调度节点, 若干 Worker 节点, 高网络 IO, 高数据库 IO	极高

选择：选择策略 2 采用 Map Reduce 框架编码软件系统。

理由：

1. 虽然 Map Reduce 实现难度较高，但它能有效分散处理压力至整个分布式集群，降低对单个节点的配置要求，特别适合处理大规模数据，显著提升系统处理效率。
2. 与单服务器多线程方案相比，Map Reduce 在面对大规模数据时表现出更高的性能提升，且整体成本相对较低，尤其在需要频繁扩展或维护的环境中更显优势。

影响：

1. 虽然单个节点硬件要求降低，但需要更多节点和更复杂的网络来支撑分布式处理和高 IO 需求。
2. 提高了系统的可扩展性，使得在数据量增加时可以通过增加节点来应对。同时，系统维护复杂度增加，需要更专业的技术支持团队。
3. 初期设备投资和人员培训费用可能较高，但随着系统的稳定运行和扩展，长期成本效益将显著。

1.4.2.2 提升数据库读写效率的备选策略

#	策略	效率提升	适用规模	性能提升描述
1	使用 Redis 数据库缓存常用数据	高	中	可大幅减少数据库负载，提高数据访问速度
2	采用数据库分片技术、优化索引和操作语句	中	大	可提高整体的并发能力和处理能力
3	升级硬件设施	不确定	大	视具体硬件升级情况而定

选择：选择策略 2 采用数据库分片技术、优化索引和操作语句。

理由：

1. 虽然 Redis 数据库的高性能缓存能显著提升数据读取速度，减少访问数据库的次数，但对于本系统，在批处理流程启动期间几乎每次都需要访问所有数据，因此缓存对常用数据的效果有限。
2. 采用数据库分片技术、优化索引和操作语句可以显著减轻各个数据库节点的负担，提高系统的并发处理能力，且几乎不增加额外开销，非常适合本系统的大规模数据环境。

3. 虽然升级硬件设施可能在一定程度上提升性能，但这种方法带来的成本较高，并且对于长远的系统可持续发展不利，因此不是最优选择。

影响：

- 1. 数据库分片可将数据分布到多服务器上，从而降低单服务器的负载，并提高查询和写入的速度。优化索引和操作语句也增强了查询效率，使得系统能够更快地处理大量数据请求。
- 2. 实施数据库分片和优化索引使得数据库架构变得更加复杂。需要进行细致的规划和配置，以确保数据均匀分布并保持高效的索引。
- 3. 虽然初期投入可能较大，包括人员培训和系统测试等，但长期来看，这种策略可以减少因性能问题导致的维护成本。同时，系统的可扩展性得到提升。
- 4. 分片可能导致跨多个数据库节点的事务处理变得复杂，需要精心设计事务处理策略来维持数据的完整性和一致性。这可能增加实施的难度，但是必要的。

1.4.2.3 引入任务调度与监控机制的备选策略

#	策略	可靠性	复杂度
1	每个节点保证完成其分配到的任务	较高	中
2	由作业调度节点保证所有任务完成	高	高

选择：选择策略 2 由作业调度节点保证所有任务完成。

理由：

- 1. 单节点调度容易导致数据一致性问题 and 单点故障风险，影响任务的完整性。
- 2. 集中作业调度节点可以实时监控和动态调整任务，优化负载分配，提高系统性能和稳定性。
- 3. 虽然策略 2 的实现复杂度较高，但可利用成熟技术有效管理。

影响：

- 1. 提高系统的整体可靠性和容错性。集中调度可即时响应各节点状态变化，减轻节点故障的后果。
- 2. 增加系统的运维复杂性，需要高效的作业调度算法和强大的硬件支持来处理集中调度的高负载。
- 3. 可能增加系统的开发和维护成本，因为需要开发和维护复杂的调度系统。

1.4.2.4 故障预防的备选策略

#	策略	资源使用	复杂度
1	定期检查系统健康，提前发现潜在问题	需要一定系统资源和时间	低
2	制定完善的灾难恢复计划 (DRP), 包括应急响应流程、灾难恢复团队和资源调配等	需要额外使用较多资源	高
3	定期进行数据备份，确保数据安全	需要一定系统资源	低
4	灾难恢复计划 + 数据备份	需要额外使用较多资源	高

选择：选择策略 4，制定灾难恢复计划并定期进行系统数据备份。

理由：

- 1. 灾难恢复计划 (DRP) 配合数据备份，提高对各类灾难（硬件故障、网络中断等）的应对能力，确保系统高可用性和数据安全。
- 2. 此组合策略能迅速恢复系统运行，减少停机时间和潜在的经济损失。

影响：

- 1. 系统的恢复能力显著提高，能在最短时间内恢复至灾难发生前的状态，保证业务连续性。
- 2. 资源消耗增加，需要定期投入额外资源用于数据备份和灾难恢复演练，可能影响整体运行效率。
- 3. 初始部署和持续维护成本提高，需要专门的团队来管理和更新 DRP 及备份策略。

1.4.2.5 故障检测的备选策略

#	策略	自动化程度	消耗资源	复杂度	实时性
1	使用监控工具 Zabbix 主动监控系统的各项指标，并设置阈值告警	高	高	高	高
2	利用 Splunk 日志工具分析系统日志，检测异常情况	中	低	中	中
3	通过心跳信号检测系统各部分的运行状态	低	低	低	中

选择：选择策略 1 使用监控工具 Zabbix 主动监控系统的各项指标，并设置阈值告警。

理由：

1. Zabbix 提供的实时监控和自动化告警最适合银行系统，确保了高可靠性和即时响应。通过灵活的告警规则，大幅减少了人工干预，提升了运维效率。
2. 虽然 Splunk 日志分析能详细记录追踪问题，但实时性不足，不符合银行系统对快速响应的需求。
3. 心跳信号方法虽然资源消耗低，但仅限于检测服务可用性，无法全面监控系统健康。

影响：

1. Zabbix 的实时告警大幅提升对故障的响应速度，对于高可用性系统至关重要。
2. Zabbix 的高级功能和持续数据处理需求导致计算和存储资源的显著增加。
3. 维护 Zabbix 系统需要专业知识，增加了维护的技术复杂度。
4. 通过阈值告警和数据分析，Zabbix 能够识别潜在问题，减少系统意外停机风险。

1.4.2.6 故障恢复的备选策略

#	策略	恢复方式	消耗资源	复杂度	恢复时间
1	发生故障时，系统自动执行重启与恢复脚本	自动	中	低	慢
2	采用备件，故障时启用备件进行恢复	手动	高	中	中
3	使用热备份服务器，故障时快速切换。	自动	高	中	快

选择：选择策略 3 使用热备份服务器，在故障时快速切换。

理由：

1. 热备份允许系统在备份期间继续运行，适合对停机时间敏感的环境。
2. 相比其他策略，热备份提供了更快的恢复时间，保证关键运营的连续性。
3. 采用备件的策略虽然有效，但涉及额外的维护与成本问题。

影响：

1. 虽然热备份服务器能快速恢复系统，但需要更高的初始和运维成本。
2. 系统的高可用性和快速故障恢复能力将显著提高整体服务的稳定性和用户满意度。
3. 需要建立和管理热备份系统的技术能力，可能需要培训或雇佣具备相关技能的专业人员。

1.4.2.7 保证日志对操作的覆盖率的备选策略

#	策略	保证方式	覆盖率	复杂度
1	对代码进行静态分析，检测是否将所有可能改变系统状态的操作都记录到了日志中	半自动	中	高
2	引入执行节点，让所有可能更改系统状态的操作都由执行节点执行并记录	自动	高	中

选择：选择策略 2 引入执行节点，让所有可能更改系统状态的操作都由执行节点执行并记录。

理由：

1. 静态分析虽然详尽但复杂性高并且无法保证完全覆盖，需要人工介入修复漏洞，影响效率和扩展性。
2. 执行节点自动记录所有状态变更操作，简化流程并提高覆盖率，优化系统扩展性和维护性。

影响：

1. 引入执行节点可能增加系统的初始复杂度和开发成本，但长期看提高了系统维护的效率。
2. 通过自动化记录，降低了人工审核和错误修正的频率，提升日常运维效率。
3. 提高了操作的可追踪性，有助于审计和监控，增强了系统的安全性和透明度。

1.4.3 步骤 4.4 确定策略与架构驱动因素之间的关系

#	方案类型	选择方案	架构驱动因素
1	提升系统资源利用率	采用 Map Reduce 框架编码软件系统	质量属性二：至多在一小时内生成并发送 200 万对账单
2	提升数据库读写效率	采用数据库分片技术、优化索引和操作语句	质量属性二：至多在一小时内生成并发送 200 万对账单
3	引入任务调度与监控机制	由作业调度节点保证所有任务完成	质量属性一：正常情况下完整完成批处理任务
4	引入故障准备机制	灾难恢复计划 + 数据备份	质量属性三：故障时记录状态并能在重启后恢复执行
5	引入故障检测机制	使用监控工具 Zabbix 主动监控系统的各项指标	质量属性三：故障时记录状态并能在重启后恢复执行
6	引入故障恢复机制	使用热备份服务器	质量属性三：故障时记录状态并能在重启后恢复执行
7	保证日志对操作的覆盖率	引入执行节点	质量属性四：将所有操作存入日志 用例三：用户可以制定系统根据处理过程生成报告 用例四：管理员可以查询、按条件筛选用户日志

1.4.4 步骤 4.5 获取初步的架构视图

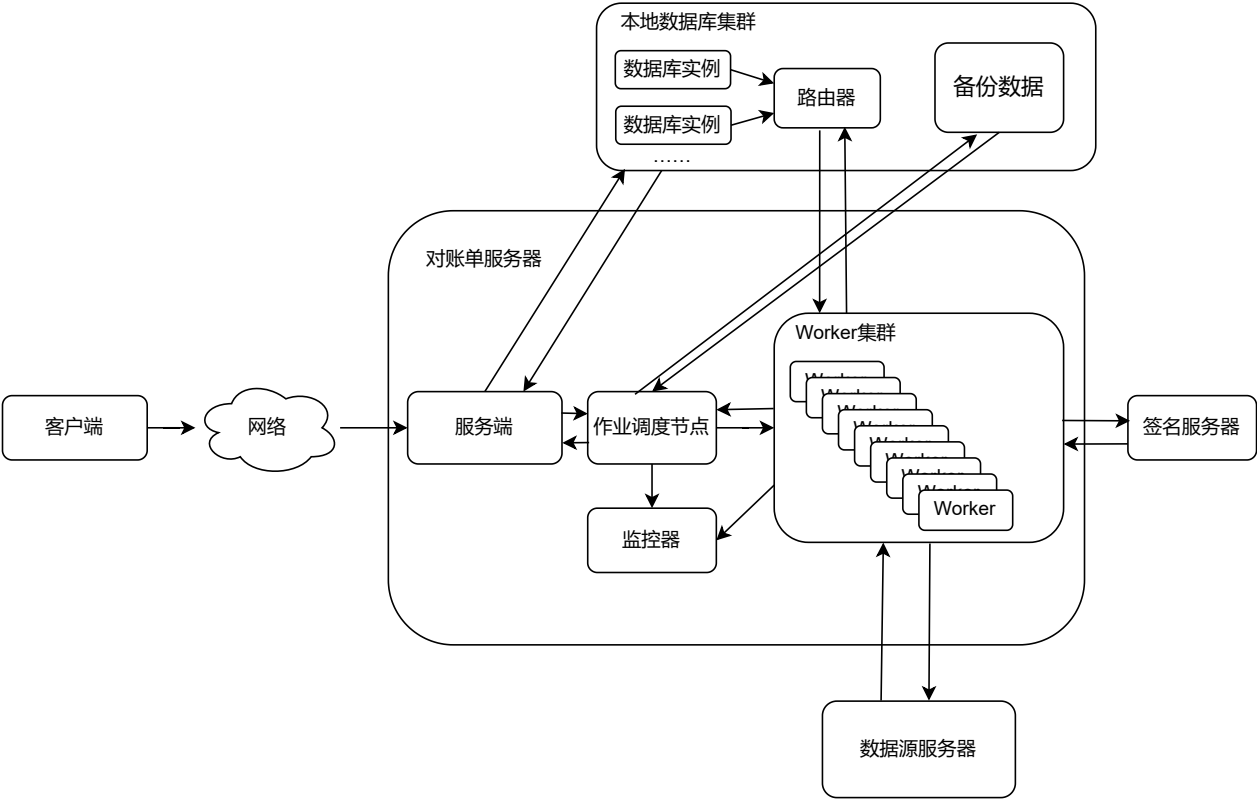
在本步骤中，我们给出初步的架构视图，包括：

- 系统要素及其开发的 ADD 迭代表
- 架构的功能视图

1.4.4.1 系统要素列表

#	系统要素	在哪次 ADD 迭代中开发
1	数据源服务器	需求
2	签名服务器	需求
3	客户端	1
4	服务端	1
5	作业调度节点	1
6	Worker 集群 (含执行节点)	1
7	监控器	1
8	数据库实例	1
9	数据库路由器	1
10	备份数据	1

1.4.4.2 初步架构视图



1.5 步骤 5: 实例化架构元素并分配职责

1.5.1 客户端

客户可以使用客户端通过网络对服务器发起请求。客户可以使用客户端进行登入，查看报告等操作。但是客户端仅可以在银行内网中使用。

1.5.2 作业调度节点

调度节点负责作业的分配与调度与监控。调度节点负责接收并管理用户提交的 MapReduce 作业。它会根据集群的资源状况来调度和分配作业。

调度节点持续监控每个任务的执行状态，包括任务的进度、成功与失败等。它会定期与工作节点进行通讯，获取任务状态信息，并记录在状态数据库中。

调度节点负责处理任务执行中的各种故障情况，例如任务失败或节点故障。当某个任务失败时，调度节点会重新调度该任务到其他可用节点重新执行。对于节点故障，调度节点会检测到并将其任务重新分配给其他健康的节点，同时持续监控故障节点的恢复进度。

1.5.3 Worker 集群节点

Worker 集群中的节点，负责执行调度节点分配给其的任务。MapJob 主要包括从数据源数据库读取数据，处理后存入本地数据库集群。ReduceJob 负责将数据库中的数据发送给签名服务器进行签名。

1.5.4 监控器

监控器监控服务器中资源使用情况以及任务完成情况。

1.6 步骤 6：为实例化的架构元素定义接口

From Element	To Element	Interface
客户端	服务端	发生请求
服务端	客户端	响应请求
服务端	数据库集群	查询数据
数据集群	服务端	返回数据
服务端	作业调度节点	发送任务
作业调度节点	服务端	返回任务状态
Worker 集群	作业调度节点	申请/完成任务
作业调度节点	Worker 集群	分配任务
Worker 集群	签名服务器	请求签名
签名服务器	Worker 集群	返回数据
Worker 集群	数据源服务器	请求数据
数据源服务器	Worker 集群	返回数据
Worker 集群	本地数据集群	保存数据或查询数据
本地数据集群	Worker 集群	返回数据
作业调度节点	备份数据	备份数据
备份数据	作业调度节点	加载数据
作业调度节点	监控器	发送状态
Worker 集群	监控器	发送状态

1.7 步骤 7：验证和完善需求，并使其成为实例化元素的约束

该迭代的所有需求都已验证。

2 迭代二

2.1 步骤 1：确认有足够的请求信息

该步骤在迭代一时已完成，迭代二直接进入步骤 2。

2.2 步骤 2：选择要分解的系统元素

在迭代二，我们选择与作业执行服务相关的元素作为要分解的系统元素，主要为作业调度节点和 Worker 集群

2.3 步骤 3：识别候选的架构驱动要素

#	架构驱动要素	重要性	难度
1	质量属性一：正常情况下完整完成批处理任务	高	中
2	质量属性二：至多在一小时内生成并发送 200 万对账单	高	高
3	质量属性三：故障时记录状态并能再重启后恢复执行	高	中
4	质量属性四：将所有操作存入日志	高	中
5	质量属性五：支持系统未来更多的使用场景和需求	高	高
6	用例一：用户可以按条件筛选、指定重新处理对账单	中	低
7	用例三：用户可以指定系统根据处理过程生成报告	低	中
8	用例四：管理员可以查询、按条件筛选用户日志	高	中

2.4 步骤 4：选择符合架构驱动的设计概念

2.4.1 步骤 4.1 识别设计关注点

设计关注点	子关注点
提升任务调度和执行的效率	提升任务并行执行能力
	优化任务分配调度
确保过程中的数据安全和权限控制	引入安全审计机制
支持系统任务量和复杂度的增加	引入模块化设计
	预留规模扩展空间

2.4.2 步骤 4.2 & 4.3 列出并选择策略

2.4.2.1 提升任务并行执行能力的备选策略

#	策略	同步、通信开销	适用场景
1	采取任务并行与传统通信协议	较高	计算密集型任务
2	采取数据并行与 Thrift 通信协议	较低	大规模数据处理任务

选择：选择策略 2 采取数据并行与 Thrift 通信协议

理由：

- 数据并行策略将整个任务划分为多个数据区域，然后将各个数据区域分派给节点集群执行，由此可以避免子任务间的依赖关系带来的同步、通信开销。适用于大规模的数据处理任务。
- 本系统主要处理大规模数据，不涉及复杂计算，因此数据并行更匹配系统需求。

影响：

- 通过减少同步和通信开销，可以显著提升任务处理速度，提高系统整体效率。
- 增强了系统的扩展性和跨语言开发的灵活性，适应更多的开发场景和未来的技术升级。
- 需要高效的数据分区机制，需要额外的算法支持和存储管理，对系统架构设计提出更高要求。

2.4.2.2 优化任务分配调度的备选策略

#	策略	任务分配效率	复杂度	适用场景
1	基于轮询的负载均衡	中	低	简单任务
2	使用 HAProxy 负载均衡器分配任务	高	中	不同任务优先级
3	使用 Apache Mesos 分布式任务调度框架	非常高	高	复杂任务

选择：选择策略 2 使用 HAProxy 负载均衡器分配任务

理由：

1. HAProxy 提供根据任务优先级和权重的动态任务分配，较基于轮询的方式更高效。
2. 与 Apache Mesos 相比，HAProxy 实现成本更低，满足当前系统对不同任务优先级的需求。

影响：

1. 提高了任务分配的实时性和效率，确保高优先级任务得到快速处理。
2. 降低了系统的整体复杂度和实施成本，提高了系统的维护性和稳定性。
3. 依赖于 HAProxy 的性能和配置，需要适时更新和调整以适应任务量的变化。

2.4.2.3 引入安全审计机制的备选策略

#	策略	安全性	实现复杂度	适用场景
1	节点使用详细日志记录	中	中	基本审计
2	节点上运行自动化审计工具	高	中	中等复杂度系统
3	将安全审计功能集成到监控服务器和执行节点中	非常高	高	高安全性需求

选择：选择策略 3 将安全审计功能集成到监控服务器和执行节点中

理由：

1. 如果采用在节点使用详细日志记录，需要有较为复杂的测试保证覆盖率，难以保证很高的安全性。
2. 如果采用在节点上运行自动化审计工具，可能会影响节点的运行性能，安全性无法保证。
3. 策略 3 通过集成审计功能，提高了安全审计的全面性和深度，降低了对执行节点性能的影响。能有效隔离并记录可能影响系统安全的操作，适合处理涉及隐私信息的高安全需求场景。

影响：

1. 提高了系统的安全性和可靠性，确保敏感数据的保护。
2. 增加了系统实施和维护的复杂性，需要专业的技术支持与持续的安全更新。
3. 可能会提高系统的初期成本，但从长远来看，能够减少安全事故带来的潜在损失。

2.4.2.4 引入模块化设计的备选策略

#	策略	耦合度	实现复杂度
1	使用消息队列进行模块之间解耦	中	低
2	使用代理节点进行模块之间解耦	低	中

选择：选择策略 1 使用消息队列进行模块之间解耦。

理由：

1. 消息队列支持模块间异步数据传递，允许模块独立扩展和部署，提高系统的灵活性和可维护性。
2. 相比代理节点，消息队列在数据耦合场景中提供更高的可靠性和扩展性，适合大规模数据处理。

影响：

1. 通过异步通信，系统可在不影响其他模块的情况下独立扩展各个组件。
2. 模块之间的低耦合度使得单独更新或优化某一部分变得更加容易。
3. 引入消息队列初期需要更多的配置和调整，但从长远来看，将为系统带来更大的灵活性和稳定性。

2.4.2.5 预留规模扩展空间的备选策略

#	策略	扩展性能	资源需求量	实现复杂度
1	使用自动伸缩计算资源，根据负载自动增加或减少实例	中	中	中
2	增加实例数量	中	高	低

选择：选择策略 1 使用自动伸缩资源进行规模扩展。

理由：

1. 自动伸缩资源能够根据实际负载动态调整，实现资源的最优配置，在需求高峰时增加实例，需求低谷时减少实例，以此达到成本效率最大化。
2. 与持续运行大量未充分利用的实例相比，自动伸缩机制减少了资源浪费和不必要的成本，尤其适用于需求波动较大的应用。
3. 自动伸缩提高了系统的响应速度和可用性，确保在用户需求增加时迅速扩展资源，而在需求减少时及时缩减，保持系统性能的稳定。

影响：

1. 通过仅在需要时增加资源，自动伸缩策略显著降低了不必要的资源开销，优化了运营成本。
2. 自动调整资源以匹配实际需求，提升了系统的整体性能和用户满意度。
3. 尽管实现自动伸缩机制需中等程度的技术投入，但长期效益明显，如成本节约和系统性能优化。

2.4.3 步骤 4.4 确定策略与 ASR 之间的关系

#	方案类型	选择方案	架构驱动因素
1	提升任务并行执行能力	数据并行与 Thrift 通信协议	质量属性二：至多在一小时内生成并发送 200 万对账单
2	优化任务分配调度	使用 HAProxy 负载均衡器分配任务	质量属性一：正常情况下完整完成批处理任务 质量属性二：至多在一小时内生成并发送 200 万对账单
3	引入安全审计机制	将安全审计功能集成到监控服务器和执行节点中	质量属性三：能够识别错误并在重新上线后继续运行 质量属性四：将所有操作存入日志 用例四：管理员可以查询、按条件筛选用户日志
4	引入模块化设计	使用消息队列进行模块之间解耦	质量属性五：支持系统未来更多的使用场景和需求
5	预留规模扩展空间	使用自动伸缩规模进行规模扩展	质量属性五：支持系统未来更多的使用场景和需求

2.4.4 步骤 4.5 获取初步的架构视图

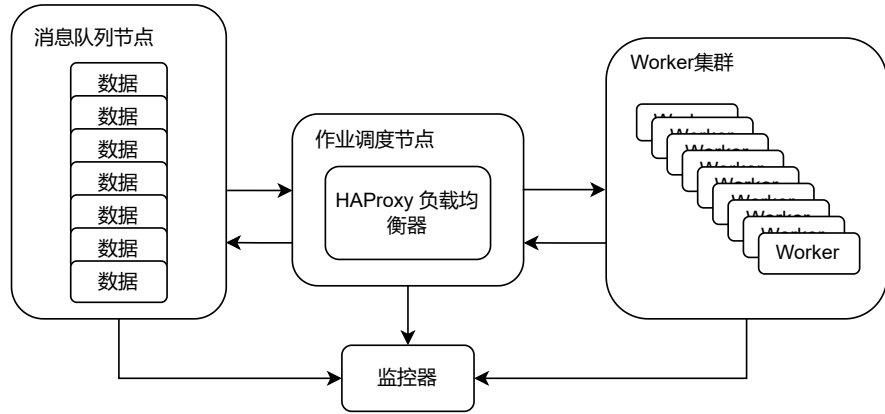
在本步骤中，我们给出与作业执行相关的系统元素的架构视图，包括：

- 系统要素及其开发的 ADD 迭代表
- 架构的功能视图

2.4.4.1 系统要素列表

#	系统要素	在哪次 ADD 迭代中开发
1	作业调度节点	1
2	Worker 集群	1
3	监控器	1
4	消息队列节点（含任务数据）	2
5	HAProxy 负载均衡器	2

2.4.4.2 初步架构视图



2.5 步骤 5：实例化架构元素并分配职责

2.5.1 作业调度节点

除迭代一中所述，作业调度节点还需要执行数据并行策略，它管理作业的调度，将接收到的作业任务划分为多个数据区域，并发送划分好的任务数据到消息队列中。

2.5.2 消息队列节点

消息队列节点接收来自作业调度节点的数据，并将其放入消息队列中，排队管理作业任务，确保任务有序处理。之后，消息队列节点将任务数据分发给 HAProxy 负载均衡器。

2.5.3 HAProxy 负载均衡器

HAProxy 负载均衡器接收来自消息队列节点的任务数据，在 Worker 集群中均衡分配作业任务，并监控 Worker 负载，动态调整任务分配。HAProxy 负载均衡器需要确保高可用性和任务处理的均衡性。

2.5.4 Worker 集群节点

Worker 集群中的节点执行分配到的作业任务，生成处理结果。此外执行节点还需要执行安全审计，确保执行过程中的数据安全和权限控制。

2.5.5 监控器

监控器需要实时监控作业调度节点、消息队列节点、HAProxy 和 Worker 集群的状态。监控服务器还需执行安全审计，记录系统运行日志和性能指标，并提供系统健康状态报告。

2.6 步骤 6：为实例化的架构元素定义接口

From Element	To Element	Interface
作业调度节点	消息队列节点	发送任务
消息队列节点	作业调度节点	返回任务状态
消息队列节点	HAProxy 负载均衡器	发送任务
HAProxy 负载均衡器	消息队列节点	返回任务状态
HAProxy 负载均衡器	Worker 集群	分配任务
Worker 集群	HAProxy 负载均衡器	申请/完成任务
Worker 集群	监控器	发送状态
作业调度节点	监控器	发送状态
消息队列节点	监控器	发送状态

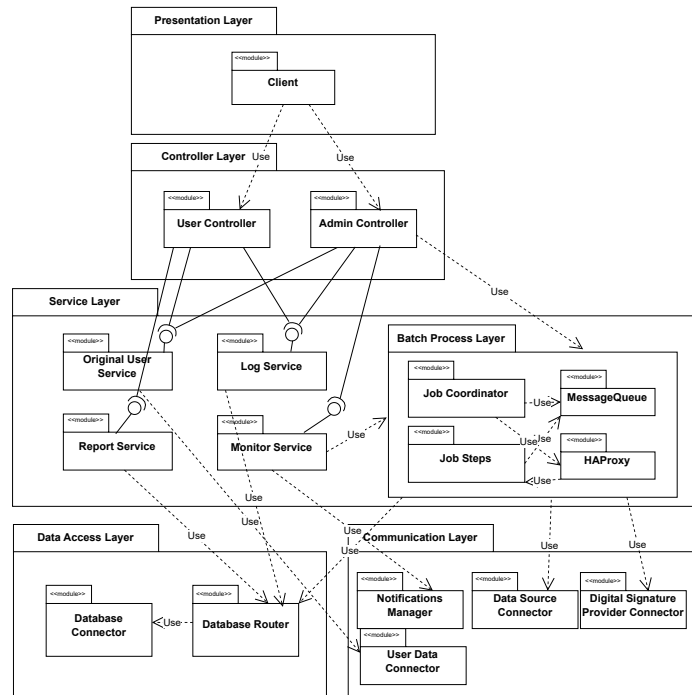
2.7 步骤 7：验证和完善需求，并使其成为实例化元素的约束

该迭代的所有需求都已验证。

Task 2: 最终软件架构文档

1 模块视图

1.1 主表示图



1.2 元素目录

1.2.1 元素与其属性

元素	职责
Client	客户端，即用户与系统的交互界面
User Controller	处理来自普通用户的请求
Admin Controller	处理来自管理员的请求
Original User Service	向原有用户服务器转发请求，处理用户相关的业务逻辑，包括用户注册、登录等功能
Log Service	记录系统操作日志，帮助监控和调试，调取数据库中的日志
Report Service	生成和管理系统的各种报告
Monitor Service	监控系统的运行状态和性能
Job Coordinator	作业调度节点，负责管理作业流程、划分数据并行区域
Job Steps	Worker 集群，负责任务流程的具体执行
HAProxy	负载均衡器，负责将作业调度节点划分的子任务分派到 Worker 集群
MessageQueue	消息队列，用于异步处理和消息传递，保证系统的高可用性和扩展性
Database Router	负责路由数据库请求，路由数据库负责数据库分片或复制的管理
Database Connector	用于连接数据库服务器，执行 CRUD 操作
User Data Connector	连接和管理外部用户数据库
Digital Signature Provider Connector	与数字签名服务器通信，处理数字签名相关业务
Notifications Manager	向外部系统发送通知，包括发生的错误等
Data Source Connector	连接提供原始数据的外部数据库服务器

1.2.2 关系与其属性

单箭头实线表示依赖，以圆圈为结束的线段为提供接口，以圆弧为结束的线段为需要接口。具体关系见图。

1.2.3 元素接口

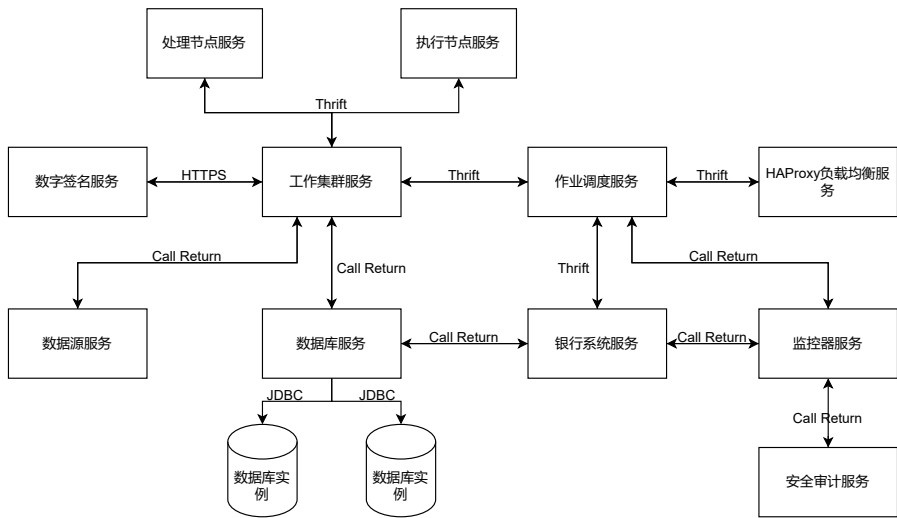
接口有四个，Original User Service 提供关于用户操作的接口，Log Service 提供日志相关的接口，Report Service 提供生成报告相关的接口，Monitor Service 提供监控数据相关的接口

1.2.4 元素具体行为

参考1.2.1

2 组件-连接器视图

2.1 主表示图



2.2 元素目录

2.2.1 元素及其属性

元素	属性
数字签名服务	提供数字签名服务，确保数据的真实性和完整性
数据源服务	提供数据源，处理数据的存储和检索
数据库服务	管理数据库实例，处理数据库的 CRUD 操作
数据库实例	存储系统需要的数据，支持数据的高效检索和存储
工作集群服务	负责协调和执行处理任务
执行节点服务	执行所有可能更改系统状态的操作
处理节点服务	进行数据的预处理
作业调度服务	调度和管理批处理作业的执行顺序和依赖关系
监控器服务	监控系统运行状态，提供系统健康检查和安全审计功能
银行系统服务	提供业务服务，处理请求
安全审计服务	提供系统安全审计服务，记录和分析系统安全事件
HAProxy 负载均衡服务	提供负载均衡服务，分发请求至不同的服务实例

2.2.2 关系与其属性

在主表示图中，单箭头表示单向服务的使用或调用关系，双箭头表示的是组件间的双向通信。关系的具体描述如下：

关系	描述
HTTP(S)	使用超文本传输协议，确保数据传输的安全
JDBC	通过 Java 与数据库连接，是系统服务与数据库服务的通信接口
Call Return	使用远程的方法调用
Thrift	使用跨语言的远程过程调用协议，支持高效的服务通信

2.2.3 元素接口

组件	接口	描述
数字签名服务	签名服务接口	接受系统生成的对账单数据为参数，等待提供方签名，然后返回完成签名的对账单。
数据源服务	外部数据访问接口	提供与外部数据源交互的功能，支持数据的读取和写入操作。
工作集群服务	集群内部节点交互接口	支持集群内部节点之间的数据和状态信息的交换，确保节点之间的协同工作。
工作集群服务	外部服务交互接口	提供工作集群与外部系统或服务交互的功能接口，支持外部请求的处理和响应。
工作集群服务	作业调度访问接口	提供对作业调度的访问和管理功能，支持作业的提交、执行和监控。
数据库服务	数据库 CRUD 接口	提供数据库的创建、读取、更新和删除操作接口，支持对数据库的常规操作。
银行系统服务	对账单系统功能访问接口	提供银行对账单系统的功能访问接口，支持对账单的生成、查询和管理。
作业调度服务	生产者访问接口	提供银行系统服务对作业调度服务的访问接口，银行系统服务通过该接口将账单数据上交作业调度服务进行处理。
作业调度服务	消费者访问接口	提供工作集群服务对作业调度服务的访问接口，工作集群通过该接口获取分发后的作业数据。
HAProxy 负载均衡服务	任务数据发布接口	接受来自作业调度服务提供的任务数据，并依据内部负载均衡功能，将任务数据进行分发，并将数据返回。
监控器服务	监控服务交互接口	提供与监控服务交互的接口，支持系统状态和性能的监控、报警和安全审计。
安全审计服务	安全审计服务功能接口	提供安全审计的功能接口，支持系统操作的记录和安全审计报告的生成。

2.2.4 元素具体行为

参考2.2.1

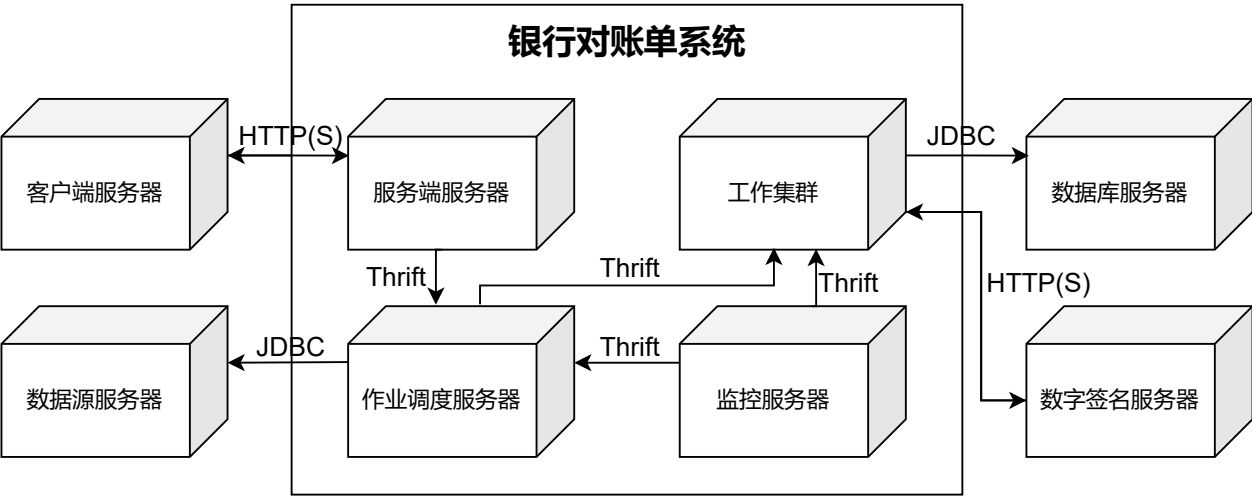
2.3 基本原理 (Rationale)

设计该视图的目的是为了清晰地展示系统中各个组件之间的交互关系和通信机制。通过该视图，我们可以一目了然地了解系统的架构层次和组件间的依赖关系。

- 1. 通过设计明确的安全通信协议（如 HTTPS）和安全审计服务，确保了数据的安全性，响应了系统的安全性需求，即质量属性四。
- 2. 组件-连接器视图清晰地定义了组件间的接口和依赖关系，能够支持系统任务量和复杂度的增加，与质量属性五相对应，即支持系统未来更多的使用场景和需求。
- 3. 视图中的监控器服务与银行系统服务、作业调度服务等相连接，通过提供监控和安全审计的功能，确保及时发现系统问题，与质量属性三相对应，即故障时记录状态并能在重启后恢复执行。同时，监控器服务记录了系统的处理过程，用户可以通过该服务组件获取系统处理报告，与用例三相对应。
- 4. 工作集群服务组件通过作业调度服务组件与 HAProxy 负载均衡服务间接连接，通过 HAProxy 负载均衡服务分配任务到不同的节点上，保证了任务处理的高效性，符合系统对性能方面的需求，即质量属性二。

3 分配视图

3.1 主表示图



3.2 元素目录

3.2.1 元素及其属性

元素	属性
数字签名服务器	提供数字签名服务，确保数据的真实性和完整性
数据源服务器	提供数据源，处理数据的存储和检索
工作集群	执行数据的相关处理工作
作业调度服务器	调度和管理批处理作业的执行顺序和依赖关系
监控服务器	监控系统运行状态，提供系统健康检查和安全审计功能
服务端服务器	处理客户端请求，提供业务服务
数据库服务器	管理数据库实例，处理数据库操作
客户端服务器	用户与系统交互的接口，提供操作入口

3.2.2 关系与其属性

关系	描述
HTTP(S)	服务器之间使用超文本传输协议通信
JDBC	系统服务器通过 Java 与数据库服务器连接
Thrift	系统内部服务器使用的跨语言的远程过程调用协议，支持高效的服务通信

3.2.3 接口属性

硬件节点（集群）	接口属性描述
服务端服务器	通过 HTTP(S) 与客户端服务器通信
数据库服务器	通过 JDBC 协议与工作集群服务器通信
数据源服务器	通过 JDBC 协议与作业调度服务器通信
作业调度服务器	通过消息队列与其他服务交互
工作集群	Kubernetes 集群内部组网，使用 thrift 协议通信
监控服务器	使用 thrift 协议通信，与其他服务交互

3.2.4 元素具体行为

参考3.2.1

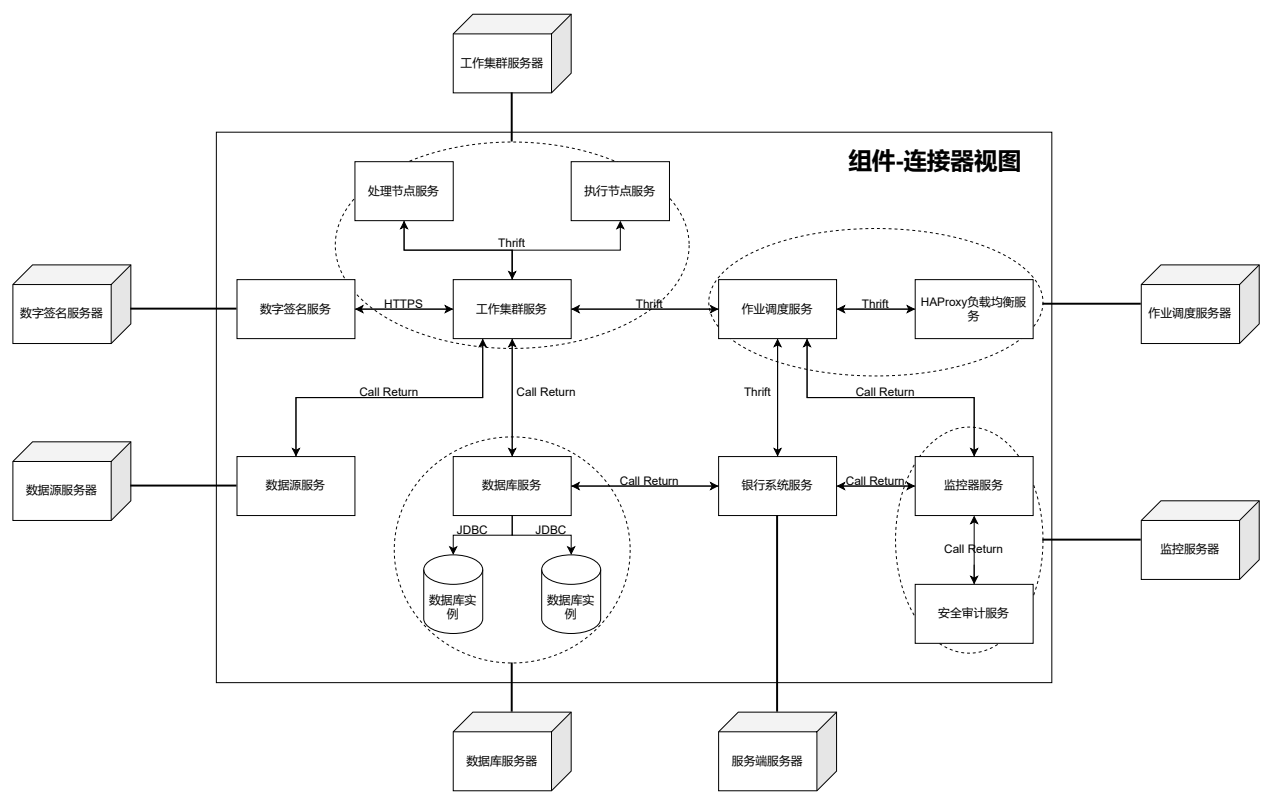
3.3 基本原理 (Rationale)

分配视图展示了系统在物理层面的布置情况，该视图的设计综合考虑了服务器拓扑结构、服务器之间的网络链接情况等。通过明确各个服务和组件的物理部署位置，可以合理分配硬件资源，优化网络通信路径。

1. 分配视图中用户服务器的使用，确保了系统能够利用现有的用户服务器资源，即限制一。
2. 为数据库服务器分配高性能存储和内存资源，确保了数据库的存储需求，同时通过 JDBC 协议与工作集群服务器连接，确保了数据访问的稳定性和可靠性，也符合限制二的要求。
3. 系统内的工作集群服务器和外部数字签名服务器之间通过 HTTP(S) 进行通信，与限制三相对应，即使用 WEB 服务与数字签名提供系统通讯。
4. 通过设计好的网络拓扑，系统内部组成网络，系统提供的 WEB 服务可以在内网中通过浏览器进行访问，符合限制四的要求。

4 交叉视图

4.1 主表示图



4.2 元素目录

本交叉视图将组件-连接器视图与分配视图结合起来，展示了系统软硬件模块之间的实际结合形式，故交叉视图中的元素即为组件-连接器视图和分配视图的元素，此处不多赘述。

4.3 基本原理

本交叉视图将组件-连接器视图与分配视图结合起来，展示了系统软硬件模块之间的实际结合形式。其中，内部为组件-连接器视图，外部则是相对应的分配视图。这样的结合，有利于将具体的软件组件与硬件相联系起来，使系统整体架构更加清晰，易于理解。交叉视图中两个视图与架构需求的对应关系均在前文给出，此处也不多赘述。

Task 3: 个人总结

1 李薛成

1.1 工作贡献

在该项目中，我担任了团队负责人的角色，主导了项目的整体进度和协调。我与张铭铭合作，负责了 ADD 方法应用报告两次迭代过程中的步骤 4-7，并参与撰写了最终软件架构文档，包括系统概览中的模块视图与分配视图部分。

1.2 个人经历与感想

首先，我深刻体验到了团队合作的重要性。在整个项目过程中，我们进行了多次长达数个小时的深入讨论。从材料的共同阅读、分析到设计的构思与执行，这一系列的合作让我感受到了集思广益的力量。这是我首次如此深入地参与讨论一个具备高容错性、各个组件都非常完整的系统设计。这种深度的讨论是我之前未曾体验过的。在这种讨论中，我不仅锻炼了我的分析和表达能力，而且在团队中自信、有理有据地表达我的想法并得到队友的认同，这让我感到非常地有成就感。

其次，从知识层面上，我也获得了巨大的收获。以往我只接触过项目手册和少数学术论文，而这次项目中参考的 CMU 技术报告和实践指导则更为实用，它们不仅详尽地介绍了被业界广泛认可的方法，也让我看到了流程标准化和科学化的重要性。在未来的软件工程工作中，我将更加注重流程的规范化和文档的重要性，以此来提高工作的效率和质量。这次经历无疑为我的专业成长提供了宝贵的资料和经验。

2 张铭铭

2.1 工作贡献

在该项目中，我作为组员参与了项目全程的工作。我与李薛成合作，负责了 ADD 方法应用报告两次迭代过程中的步骤 4-7，并负责文档中多个架构视图的设计和绘制工作，完成了最终软件架构文档中组件-连接器视图的主表示图与元素目录，关系与元素接口。

2.2 个人经历与感想

刚开始接触到 ADD 方法时，我一头雾水，不清楚为什么要使用 ADD 这样繁琐的方法，但还是按部就班地根据 CMU 的文档和相关要求识别架构驱动需求，选择设计策略。当我完成了这部分工作，准备开始设计架构视图时，我才豁然开朗：设计系统架构时，我们可以将系统架构进行细分，从更为抽象的架构需求出发，一步步细化为设计关注点、设计架构策略，最终形成的一个个策略就组成了我们系统的架构，这些架构策略很容易就可以串联起来，并形成我们想要的架构。

经过本次作业，我深刻体会到，ADD 等方法的存在目的是为了更加系统规范合理的进行软件架构的设计。软件工程之所以成为工程，是因为具有着相当严格和详细的文档规范与编写原则。一个好的系统离不开大量文档的编写，从需求到设计再到架构，一步一步确定出软件的轮廓。我深刻地感受到，软件架构设计是一个复杂的过程，但复杂的同时也能通过不断的拆解组合，变成一个个更细粒度更简单的任务，让我体验到了软件架构设计的艺术。

3 陈皓鑫

3.1 工作贡献

在该项目中，我作为组员参与了项目全程的工作。主要负责了迭代一、迭代二的列出策略与选择，设计了架构视图与设计架构元素，并完设计其接口。在最终软件架构文档中，我完成了最终软件架构文档中模块视图的主表示图与元素目录，关系与元素接口，并对其他部分参与讨论修正。

3.2 个人经历与感想

采用 ADD 方法的设计是一种从上而下、追求卓越的过程。它使我们更加关注质量属性场景，从中出发，识别合适的架构模式，然后逐步分解，以满足需求和质量属性的规定。这种方法将复杂问题具体化、简化为多个小问题，是一个迭代过程，每一步的细化都基于之前步骤的反馈和评估。这种迭代有助于逐步完善架构设计，确保我们能够在每一阶段都做出最优的设计决策。

通过这次实践，我更加明白了架构设计的重要性，以及它在整个软件开发过程中的核心地位。在未来的项目中，我将更加重视架构设计的前期工作，利用 ADD 方法来确保设计的合理性和系统的稳定性。同时，我也认识到，一个好的架构设计师不仅需要具备技术知识，更需要有解决问题的能力 and 视野，能够从宏观上把握项目的发展方向。

这种深度的思考和实践让我对软件工程有了更深刻的认识，也为我未来的职业生涯奠定了坚实的基础。我期待在未来能够将这些理论知识转化为实际的能力，参与到更多高质量的软件项目中，为用户和社会创造更多价值。

4 张哲恺

4.1 工作贡献

在该项目中，我作为组员参与了项目全程的工作。我分析材料提取出要求的功能需求、质量属性和约束等，识别候选的架构驱动元素，并修正一些不准确的非功能性需求识别和替代策略的表达。在最终软件架构文档中，我主要负责了分配视图和交叉视图部分，并对其他部分参与讨论修正。

4.2 个人经历与感想

在本次作业中，我首先阅读了老师的 PPT，查阅了一些参考资料，并仔细阅读了 CMU 的文档。在完成设计的过程中，我对 ADD 方法的认识也在不断深化，也对质量属性、设计策略等元素有了全新的认知。另外，我也提升了从文档中提取信息的能力，形成了更有层次、有体系的软件设计方法论。ADD 方法不仅仅是一个机械的步骤执行过程，而是需要架构师深刻理解系统需求、质量属性和各种设计策略。

我认为，ADD 方法的一个主要难点在于项目初期对需求、质量属性和约束的准确识别，以及对架构元素和设计关注点的精确把握。这些因素是整个架构设计的基石，只有在这一阶段进行彻底的讨论和明确，才能有效地推动项目的后续开展，避免在开发过程中因为设计理解的模糊而进行大量的修改。在团队合作中，这需要每位成员展开充分的思维，细致地分析项目要求，考虑所有可能的场景。ADD 的逐步细化过程使得架构设计变得更加可控，同时也能及时发现并解决问题。通过这次实践，我对软件架构的设计方法有了更深的理解，也更清晰地认识到软件架构设计的重要性，这对我未来的职业生涯和学术研究都具有重大的指导意义。